Predicting Coronary Heart Disease from BRFSS Survey Data with Class-Weighted Logistic Regression

Leo Descombes 355583, Hector Mir 347321
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict whether a respondent of the Behavioral Risk Factor Surveillance System (BRFSS) survey has had a myocardial infarction or angina (MICHD) from 321 self-reported features. The task is strongly imbalanced (8.8 % positives) and the data contains many missing values and special survey codes. We combine a train-only cleaning and preprocessing pipeline with an L2-regularized logistic regression that weights the positive class, and we tune the decision threshold to maximize the F1 score. In 4-fold stratified cross-validation the model reaches an F1 score of 0.428, and 0.438 on the AIcrowd held-out test set. The F1 score is almost insensitive to the regularization strength and the class weight, because the tuned threshold compensates for the class weight.**

I. Introduction

The training set contains $N = 328{,}135$ respondents and the test set $109{,}379$, each described by $D = 321$ features: raw survey answers and variables computed by the survey (prefixed by an underscore). Only $8.8\,\%$ of the training respondents are positive. A classifier that always predicts "no MICHD" therefore reaches $91.2\,\%$ accuracy but an F1 score of 0. We thus select and evaluate our models with the F1 score of the positive class.

We propose a train-only preprocessing pipeline and a class-weighted logistic regression with a tuned threshold, implemented with NumPy only. Once the threshold is tuned, neither the hyperparameters nor the preprocessing variants change the F1 score by more than 0.001.

II. Models and Methods

A. Methods implemented in class

We implemented the six methods seen in the course labs, adapted from our lab solutions: linear regression with gradient descent and with stochastic gradient descent (MSE loss), least squares and ridge regression (both solved with the normal equations), and logistic regression with and without L2 regularization (gradient descent). They are provided in `implementations.py`. Our final model, described below, extends regularized logistic regression with class weights, accelerated gradient descent and a tuned decision threshold, to address the class imbalance.

B. Cleaning of survey codes

BRFSS encodes non-answers with special values. "Don't know" and "refused" codes (7/9, 77/99, 777/999) become missing values, and "none" (88) becomes 0 in "number of days" columns; 7 is kept as a valid category when a column also contains 8. The rules are chosen from the training set and applied identically to the test set.

C. Preprocessing

All statistics below are computed on the training data only.

- Columns with more than $80\,\%$ missing values or zero variance are dropped (200 of 321 columns are kept).
- Missing values are imputed with the column median, and all kept columns are standardized.
- Columns with 3 to 15 distinct values are additionally one-hot encoded. Levels with a frequency below $1\,\%$ are ignored, and the first level is used as reference (201 columns).
- For each kept column with at least $5\,\%$ missing values, a binary indicator of missingness is added (106 columns), since not answering a question may itself be informative.

With a bias column, the final design matrix has 508 columns.

D. Model

With labels $y_n \in \{0, 1\}$, we minimize the class-weighted, L2-regularized negative log-likelihood

$$\mathcal{L}(w) = \frac{1}{N} \sum_{n=1}^{N} s_n \Big[\log\left(1 + e^{x_n^\top w}\right) - y_n \, x_n^\top w \Big] + \lambda \|\tilde{w}\|^2, \quad (1)$$

where $s_n = \rho$ for positive samples and 1 otherwise (then normalized to mean 1), and $\tilde{w}$ excludes the bias. We minimize (1) with Nesterov-accelerated gradient descent for 400 iterations, with step size $1/L$, where $L = \frac{1}{4}\lambda_{\max}\left(\frac{1}{N} X^\top S X\right) + 2\lambda$ is the Lipschitz constant of the gradient, estimated by power iteration. A sample is predicted positive if $\sigma(x_n^\top w) \geq t$.

E. Model selection

We use 4-fold stratified cross-validation. Cleaning and preprocessing are refitted on the training folds of each split, so no statistic of the validation fold is used for training. We perform a grid search over $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$ and $\rho \in \{2, 3, 4, 6\}$. For each fold, the threshold t maximizing the validation F1 is selected on a grid from 0.05 to 0.95 (step 0.01), and the final threshold is the mean over the folds. The final model is retrained on the whole training set with the selected hyperparameters.

III. Results

TABLE I
CROSS-VALIDATED F1 SCORE (MEAN OVER 4 FOLDS) FOR EACH REGULARIZATION STRENGTH λ AND POSITIVE-CLASS WEIGHT ρ. STANDARD DEVIATIONS OVER FOLDS ARE AT MOST 0.0011.

	$\rho = 2$	$\rho = 3$	$\rho = 4$	$\rho = 6$
$\lambda = 10^{-5}$	0.4278	0.4276	0.4275	0.4273
$\lambda = 10^{-4}$	0.4280	0.4278	0.4280	0.4275
$\lambda = 10^{-3}$	0.4284	0.4284	0.4282	0.4281
best t	0.33	0.43	0.50–0.51	0.60–0.61

TABLE II
ABLATION OF THE PREPROCESSING CHOICES (4-FOLD STRATIFIED CV, $\lambda = 10^{-4}$, $\rho = 4$). MEAN $\pm$ STANDARD DEVIATION OVER FOLDS.

Preprocessing	F1	Accuracy
One-hot ≤ 6 levels, no indicators	0.4273 ± 0.0014	0.8714
+ missingness indicators	0.4275 ± 0.0017	0.8707
One-hot ≤ 15 levels, no indicators	0.4280 ± 0.0007	0.8698
Both (final)	0.4280 ± 0.0005	0.8720

A. Hyperparameters

Table I shows that all 12 configurations reach an F1 score between 0.4273 and 0.4284, a range comparable to the fold-to-fold standard deviation. We select $\lambda = 10^{-3}$, $\rho = 2$ and $t = 0.3325$, which gives a cross-validated F1 score of 0.4284 ± 0.0011 and an accuracy of 0.871.

B. Preprocessing

Table II shows that extending the one-hot encoding to 15 levels increases the F1 score from 0.4273 to 0.4280, while the missingness indicators add at most 0.0002. Both gains are smaller than the fold-to-fold standard deviation. We keep both options, as the combination gives the same F1 score as the extended one-hot encoding alone with the lowest variance across folds.

C. Test set

On AIcrowd, the final model obtains an F1 score of 0,438 and an accuracy of 0,874. Our first submission, without missingness indicators and with one-hot encoding up to 6 levels, obtained an F1 score of 0.437 and an accuracy of 0.875. The model predicts $13.7\,\%$ of the test respondents as positive.

D. Most influential features

Among standardized features, the largest weights are those of sex, general health, age (`_AGE80`, `_AGEG5YR`), history of stroke (`CVDSTRK3`), high cholesterol (`TOLDHI2`, `_RFCHOL`), cholesterol check (`CHOLCHK`), COPD (`CHCCOPD1`) and pneumonia vaccination (`PNEUVAC3`). Since BRFSS codes "yes" as 1 and "no" as 2, a negative weight on these questions means that answering "yes" increases the predicted risk.

IV. Discussion

The class weight ρ and the threshold t play the same role: weighting the positives mainly shifts the predicted probabilities upwards, and the tuned threshold rises accordingly (from 0.33 to 0.60 in Table I), so the final F1 score barely changes. Tuning the threshold alone is therefore sufficient for this model. Similarly, the regularization strength has little effect, as N is much larger than the number of features. The small standard deviations across folds show that the cross-validated estimates are reliable.

The main limitation is the linear model: the decision boundary cannot capture interactions between risk factors, such as age and sex. In addition, our cleaning rules are heuristics based on the observed values rather than on the BRFSS codebook, and median imputation ignores the dependencies between features. Coding each variable from the codebook and adding interaction features are the most promising improvements.

V. Summary

Our class-weighted logistic regression reaches a cross-validated F1 score of 0.428 and 0.437 on the AIcrowd test set. Tuning the decision threshold is essential, while the other choices have a negligible effect.

Use of AI tools

We used an AI assistant (Claude) throughout the project. It was especially useful to debug our code and to deepen our understanding of the course concepts. It also helped us draft parts of the pipeline and of this report, which we reviewed, ran and validated ourselves.